

INGENIERÍA COMPUTACIONAL Y SISTEMAS INTELIGENTES

Seguimiento de color

Robótica Inteligente

Estudiantes: Eneko Perez, Alan García y Alex Amenabar

Índice

1. Análisis del problema	2
2. Estrategia utilizada y justificación	2
3. Algoritmo e implementación	2
4. Evaluación del comportamiento	5
5. Mejoras posibles	5

1. Análisis del problema

En este proyecto se aborda el problema de implementar un módulo de comportamiento para un robot móvil que le permita seguir un objeto de un color determinado. Para ello, el robot debe utilizar los datos proporcionados por la cámara para encontrar áreas con el color que se quiera seguir, adaptando la velocidad lineal y la velocidad angular en función de la distancia y posición relativa del objeto identificado.

2. Estrategia utilizada y justificación

El robot utiliza las imágenes captadas por una cámara para identificar la posición del objeto que debe seguir. La estrategia implementada consiste en orientar al robot de manera que el objeto a seguir permanezca siempre en el centro de la imagen capturada por la cámara, ajustando para ello tanto la velocidad lineal como la angular. La velocidad lineal se ajusta en función de la distancia con el objeto, y la velocidad angular, en función de la posición del centroide del objeto a seguir respecto al centro de la cámara.

Procesando las imágenes proporcionadas por la cámara, es posible obtener las coordenadas (x, y) en la imagen correspondientes al centroide del color ajustado. Estas coordenadas relativas de la posición del objeto con respecto al robot se emplean para ajustar su velocidad de rotación. La velocidad de rotación es proporcional a la posición del centroide, de manera que si el objeto se encuentra muy cerca del centro de la imagen, la velocidad angular será baja; mientras que si se encuentra en un extremo, la velocidad será mayor. De esta manera, se consigue un movimiento angular reactivo del robot que permite mantener la cámara «apuntando» hacia el objeto. Además, en el caso de que el robot estuviera siguiendo un objeto y este haya salido del campo de visión de la cámara, el robot girará en la dirección en la que estaba el objeto antes de salir de su campo de visión, permitiendo una posible relocalización del objeto.

Finalmente, para calcular la velocidad lineal se han implementado dos estrategias. La primera estrategia consiste en utilizar el área que ocupa el objeto en la imagen proporcionada por la cámara. La idea es que la velocidad lineal del robot sea inversamente proporcional al área en la imagen del objeto a seguir. Así, si el área es grande, el objeto debe de estar cerca haciendo que el robot reduzca la velocidad y, por el contrario, si es pequeña el robot debe acelerar.

Sin embargo, esta estrategia no es la más adecuada ya que presenta algunas limitaciones importantes: el sistema debe de ser ajustado para el área visible del objeto lo que lo hace muy sensible a cambios de perspectiva o oclusiones, que son bastante comunes y están ligadas también al ruido del sistema de detección del objeto. Por ello, la segunda opción es utilizar la estimación de profundidad dada por la cámara del robot y aplicar una estrategia similar: la velocidad lineal es directamente proporcional a la distancia del objeto.

3. Algoritmo e implementación

La implementación de este proyecto se encuentra en este repositorio de GitHub.

En la Figura 1 se muestra la estructura de nodos, mensajes y servicios utilizada en la implementación.

3 ALGORITMO E IMPLEMENTACIÓN

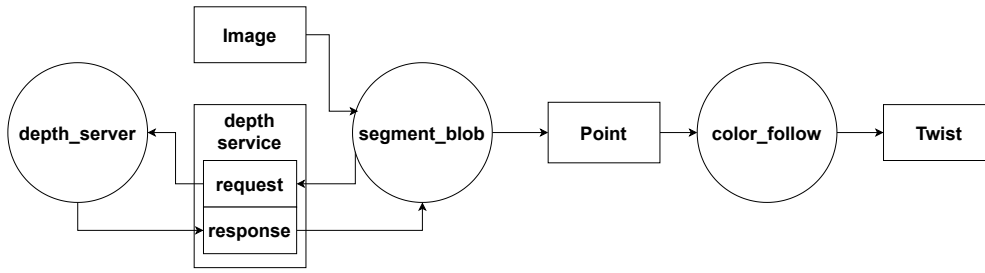


Figura 1: Estructura de nodos, mensajes y servicios de la implementación.

El nodo *segment_blob* es el encargado de procesar la información de la cámara. Recibe como entrada un mensaje de tipo *Image*, que contiene la imagen RGB; y envía como salida un mensaje de tipo *Point*, que contiene la información de salida codificada en las componentes x , y y z donde:

- x indica la posición normalizada del objeto en el eje X , situando el origen en el centro de la imagen.
- y codifica la información de la distancia al objeto, ya sea mediante el área normalizada o con la estimación de profundidad (parámetro elegible por el usuario).
- z codifica un *flag* que indica si se está detectando un objeto o no.

La detección de objetos por color se realiza empleando el Algoritmo 1, que hace uso de funciones de *OpenCV*. Una vez se ha localizado el centroide del objeto, el nodo *segment_blob* codifica la información en un mensaje de tipo *Point* y la envía al nodo de control de acuerdo al Algoritmo 2.

Algorithm 1 Encontrar centroide

```

1: function FINDCENTROID(src, params)
2:   Definir lowRGB y highRGB
3:   img_segmented ← Umbralizar src entre lowRGB y highRGB
4:   img_segmented ← applyErodeAndDilate(img_segmented)
5:   contornos ← findContours(img_segmented)
6:   if no hay contornos then
7:     return valores por defecto
8:   end if
9:   Dibujar contornos en src
10:  Inicializar listas para contornos y bounding boxes
11:  for cada contorno do
12:    Aproximar contorno y calcular bounding box
13:  end for
14:  Encontrar el contorno más grande
15:  if contorno más grande válido then
16:    Calcular centroide con momentos
17:  end if
18:  return centroide, imagen segmentada y datos de bounding box
19: end function

```

En el caso de emplear la estrategia de estimación de distancia con el mapa de profundidad de la cámara, este nodo se comunicará con el servicio *depth_service* utilizando la interfaz *GetDepth.srv*. De esta manera, el nodo *segment_blob* se comportará como un cliente que envía un bounding box y el servidor responderá con la media y desviación típica de la distancia estimada. Para ello, el

3 ALGORITMO E IMPLEMENTACIÓN

cliente debe esperar a que lleguen las respuestas 2.

Algorithm 2 SegmentBlob Algorithm

```

1: use_depth_map                                     ▷ Flag establecido por el usuario
2: cresx, cresy, img, numc, bbox_area, x0, y0, x1, y1 = findCentroid
3: is_centroid = 1 if cresx != -1 else 0
4: normalize bbox_area
5: normalize(px, py)
6: y_comp ← bbox_area
7: if use_depth_map then
8:   if is_centroid == 1 then
9:     request(x0, y0, x1, y1)
10:  end if
11:  depth, std_dev = Check_resposes()
12:  y_comp ← depth
13: end if
14: Point p = (px_norm, y_comp, is_centroid)
15: Publish(p)

```

Finalmente, el nodo *color_follow* recibe el mensaje del nodo *segment_blob* y lo decodifica para calcular las comandas de control del robot, enviando un mensaje de tipo *Twist* con las velocidades lineales y angulares adecuadas. Dependiendo de si el usuario ha decidido emplear la estrategia del cálculo de la velocidad lineal mediante el área o no se utilizarán los algoritmos 3 o 4 respectivamente. En ambos casos, el giro del robot es reactivo a la posición del objeto detectado en el eje *X* y si no se encuentra un objeto ($z = 0$) el la velocidad lineal del robot será nula y éste girará hacia la última dirección registrada del objeto.

En el caso de emplear la estrategia del área del objeto, si ésta es mayor que 0.2, el objeto ocupa una parte considerable de la imagen, y por tanto el robot no se mueve muy rápido. Si no es así, el robot se mueve a una velocidad inversamente proporcional al área. En el caso contrario, cuanto menor sea la distancia, más cerca está el objetivo y por lo tanto más lento se mueve el robot.

Algorithm 3 Cálculo de acciones utilizando el área

```

1: function COMPUTE_ACTIONS(rotación, es_objetivo, área)
2:   target_speed ← 0,0
3:   target_rotation ← -2,0 × última_dirección
4:   if no hay objetivo (es_objetivo = false) then
5:     return (target_speed, target_rotation)
6:   end if
7:   última_dirección ← 1,0 si rotacion > 1,0, sino -1,0
8:   if rea > 0,2 then
9:     target_speed ← 0,25
10:    target_rotation ← -2,0 × última_dirección
11:    return (target_speed, target_rotation)
12:  end if
13:  if rea ≤ 0,0 then
14:    rea ← 0,001
15:  end if
16:  target_speed ←  $\frac{1,0 - \text{area}}{1,5}$ 
17:  target_rotation ← -1,0 × rotacion × 2,0
18:  return (target_speed, target_rotation)
19: end function

```

Algorithm 4 Cálculo de acciones con estimacion de profundidad

```

1: function COMPUTE_ACTIONS_WITH_DEPTH(rotación, es_objetivo, dist)
2:   target_speed  $\leftarrow$  0,0
3:   target_rotation  $\leftarrow$   $-2,0 \times \text{última\_dirección}$ 
4:   if no hay objetivo (es_objetivo = false) then
5:     return (target_speed, target_rotation)
6:   end if
7:   última_dirección  $\leftarrow$  1,0 si rotacion > 1,0, sino  $-1,0$ 
8:   if No hay distancia computada, dist < 0 then
9:     target_speed  $\leftarrow$  0,2
10:    target_rotation  $\leftarrow$   $-2,0 \times \text{última\_dirección}$ 
11:    return (target_speed, target_rotation)
12:  end if
13:  if dist < 1,0 then
14:    target_speed  $\leftarrow$  0,1
15:    target_rotation  $\leftarrow$   $-2,0 \times \text{última\_dirección}$ 
16:    return (target_speed, target_rotation)
17:  end if
18:  target_speed  $\leftarrow$   $2,0 \times \text{dist}$ 
19:  target_rotation  $\leftarrow$   $-1,0 \times \text{rotacion} \times 2,0$ 
20:  return (target_speed, target_rotation)
21: end function

```

4. Evaluación del comportamiento

El comportamiento del robot se puede observar en este video. En general se observa el comportamiento que se ha descrito a lo largo del documento.

Cuando el objeto está lejos, el robot va un poco más rápido para no perderlo de vista. A su vez, cuando el objeto sale del campo de visión del robot, este sigue girando en la dirección en la que lo había visto por última vez, y la velocidad angular también varía un poco en función de la distancia del centroide con el centro de la imagen captada por el robot.

5. Mejoras posibles

La implementación actual puede ser mejorada de varias maneras. Para empezar, el cálculo de la velocidad lineal utilizando el área ocupado por el objeto como medida de distancia no es del todo adecuado, ya que no tiene en cuenta el tamaño real del objeto. Es decir, un objeto puede ocupar gran parte del campo de visión de la cámara porque es grande, aunque esté lejos. En esos casos el comportamiento del robot no sería del todo correcto.

Una posible opción para mejorar esto es estimar la profundidad utilizando la proporción de tamaños. Si se conoce el tamaño real del objeto y cómo varía su tamaño en la imagen en función de la distancia, es posible emplear la Ecuación 1 para calcular la distancia del objeto a la cámara.

$$d = \frac{f \cdot h_{\text{real}}}{h_{\text{imagen}}} \quad (1)$$

En esta formula, se calcula la distancia (d) del objeto a la cámara, con la distancia focal de la cámara (f), la altura real del objeto (h_{real}) y la altura del objeto en la imagen (h_{imagen}). Este

5 MEJORAS POSIBLES

método es efectivo cuando el objeto se encuentra de frente a la cámara y su orientación no varía significativamente. En el caso de la cartulina, esto no representa un problema. Sin embargo, si el objeto se inclina o su altura varía en función del ángulo de visión, la precisión del cálculo puede verse afectada.